
TP 6 (programmation) - Matrices et listes multi-dimensionnelles.

Comme d'habitude, vous devez rendre votre TP dans le délai indiqué. Lorsqu'un fichier contenant les doctests est fourni, vous devez le compléter. Sinon, vous devez écrire un programme par exercice, avec comme nom `exoX.py` (X est le numéro de l'exercice) et mettre en commentaires les tests que vous avez faits pour vérifier que votre programme fonctionne.

Comme expliqué dans la partie algorithmique du cours, les listes peuvent elles-mêmes contenir des listes de n'importe quelle longueur ; on parlera de *matrice* dans le cas où les sous-listes ont toutes la même longueur.

Exercice 1

Complétez le fichier `exo1.py` fourni et testez vos fonctions :

- en vérifiant avec les doctests (attention, cela ne garantit pas que votre code est correct !),
- et en utilisant vos fonctions dans la partie `main` du programme.

Autant que possible, évitez de réécrire plusieurs fois un code qui fait la même chose. Les fonctions sont réutilisables d'une question à l'autre. Vous pouvez ajouter vos propres fonctions auxiliaires.

Attention, vos fonctions ne peuvent pas modifier les listes passées en paramètre.

- (a) Déclarez la matrice `t1` suivante et affichez-la (bien alignée, sans les bordures) :

8	3	3	3
5	6	1	5
0	7	9	1

- (b) Remplacez la première case de la dernière ligne de `t1` par un nombre aléatoire compris entre 1 et 9 et affichez `t1` pour vérifier.
- (c) Déclarez une matrice `t2` de taille 5×7 remplie d'entiers aléatoires entre 0 et 10 (exclu) puis affichez-la.
- (d) Écrivez une fonction `cherche` qui prend une matrice et un élément en arguments et renvoie `True` si cet élément se trouve dans la matrice et `False` sinon.
- (e) Écrivez une fonction `somme_colonnes` qui prend une matrice en argument et renvoie la liste des sommes de ses colonnes.
- (f) Écrivez une fonction `annule_diagonales` qui prend une matrice en argument. Si elle possède autant de lignes que de colonnes, la fonction renvoie une nouvelle matrice contenant les mêmes éléments sauf sur les diagonales qui contiennent désormais des zéros. Sinon, elle renvoie simplement la matrice.
- (g) Écrivez une fonction `compacte` qui prend deux matrices en arguments. Si elles ont les mêmes dimensions, elle renvoie une matrice de mêmes dimensions contenant les couples formés par les éléments de chaque matrice aux mêmes positions, chaque couple étant représenté par une liste de deux éléments. Sinon, elle renvoie `None`.
- (h) Écrivez une fonction `cherche_somme_voisins` qui prend une matrice en argument et qui cherche s'il existe un élément qui est la somme des ses voisins (comme le premier élément de `t1`, par exemple). Dans ce cas, elle renvoie sa position et sinon, `None`.
Remarque : tout élément a entre 2 et 4 voisins (on ne prend pas en compte les diagonales).
- (i) ★ Écrivez une fonction `decalage_cyclique` qui prend une matrice en argument et en renvoie une nouvelle contenant les mêmes éléments sauf sur les bords où ils sont

décalés cycliquement vers la droite. Par exemple, pour `t1`, on obtient :

5	8	3	3
0	6	1	3
7	9	1	5

Exercice 2

Complétez et testez le fichier `exo2.py`.

Attention, vos fonctions ne peuvent pas modifier les listes passées en paramètre.

- (a) Écrivez une fonction `separe` qui prend une liste `lst` en paramètre et renvoie deux listes :

- une qui contient les premier, troisième, cinquième, ... éléments de `lst` et
- une qui contient les autres.

Par exemple :

```
>>> separe([0, 2, 1, 4, 5])  
[[0, 1, 5], [2, 4]]
```

Conseil : essayez d'abord avec une liste de longueur paire et vérifiez ensuite si ça fonctionne quand c'est impair (sinon, corrigez votre programme).

- (b) Écrivez une fonction `combine` qui prend deux listes d'entiers `lst1` et `lst2` en paramètres et renvoie une liste contenant le premier élément de `lst1`, suivie du premier élément de `lst2`, puis le deuxième élément de `lst1`, et ainsi de suite. Par exemple :

```
>>> combine([0, 1], [2, 3, 4, 5])  
[0, 2, 1, 3, 4, 5]
```

Conseil : essayez d'abord avec des listes de même longueur.

- (c) Écrivez une fonction `debut_croissant` qui prend une liste d'entiers `lst` en paramètre et renvoie le plus long préfixe croissant de `lst`, c'est-à-dire la plus longue sous-liste commençant au début de `lst` et ne contenant pas de décroissance.

Par exemple :

```
>>> debut_croissant([1, 1, 1, 2, 9, 4, 2, 2])  
[1, 1, 1, 2, 9]
```

- (d) Écrivez une fonction `decoupage_croissant` qui prend une liste d'entiers `lst` en paramètre et renvoie la liste des sous-listes maximales croissantes de `lst`. Attention, on souhaite que la **complexité de cette fonction soit linéaire**. Par exemple :

```
>>> decoupage_croissant([1, 1, 1, 2, 9, 4, 2, 2])  
[[1, 1, 1, 2, 9], [4], [2, 2]]
```